Gallery Greeters – Final Report

41069 - Robotics Studio 2 *AUT 25*

Dylan Purbrick - 24945343

Executive Summary

The Gallery Greeters project, developed as part of the Robotics Studio 2 subject at UTS, aimed to deliver a fully autonomous robotic system to host guests at a simulated art gallery. The robot was expected to greet guests, guide them to tables or artworks, take drink orders, and provide artwork-related information through voice interaction and vision recognition.

This project was completed in collaboration with Anika Roth, Andrew Coorey, and Lachlan Scott Rogers. Together, we designed, implemented, and tested a multi-subsystem ROS2-based platform using a TurtleBot3 Waffle Pi across four structured sprints. The system included voice command processing, path planning, robot control, and visual perception subsystems.

My primary responsibilities included leading the Perception subsystem, developing a real-time artwork recognition node using ORB-based template matching, and serving as project manager, coordinating team progress, technical documentation, and final deployment.

At handover, the system was fully integrated and demonstrated in simulation. All robot behaviours were successfully tested on hardware in the days leading up to demo day, although unforeseen technical issues limited the live demonstration. Despite this, the system met its design goals and is reproducible by others via the final documentation and GitHub repository. Upon handover, I had achieved a Credit-level implementation of the Perception subsystem, having not yet integrated Machine Learning or a Neural Network for recognition.

This project significantly deepened my experience with ROS2, computer vision, system integration, and collaborative development under time-constrained, high-stakes conditions.

Contents

Executive Summary ii
Introduction 1
Sprint Breakdown & Contribution 2
Sprint 1 – Foundation & Planning 2
Sprint Goals 2
Group Deliverables 2
Individual Contributions 2
Client & Coach Feedback 2
Response to Feedback 2
Sprint 2 – Development & Testing Setup . 3
Sprint Goals 3
Group Deliverables 3
Individual Contributions 3
Client & Coach Feedback 3
Response to Feedback 3
Sprint 3 – Integration & Validation 4
Sprint Goals 4
Group Deliverables 4
Individual Contributions 4
Client & Coach Feedback 4
Sprint Demonstration Video 4
Sprint 4 – Documentation & Handover ... 5
Sprint Goals 5
Group Deliverables 5
Individual Contributions 5
Pre-Demo Testing and Functionality ... 6
Handover & Demo Day Issues 6
Client & Coach Feedback 7
Response to Feedback 7
Perception Subsystem Technical Description 8
Purpose 8
Key Interfaces 8
Source Files 8
Subscribed Topics 8
Published Topics 8
Service Topics 8
Published Files (For Debugging Purposes) 8
Configurable Settings 8
Node description & algorithm logic 9
Perception System Architecture 10
Team Collaboration 11
Reflection & Individual Development 12
Technical Skills 12
Project Management 12
Soft Skills 12
Key Lessons 12
Conclusion 13
Appendix 14
Appendix A: Group Contract 14
Appendix B: Risk Assessment 21
Appendix C: Perception Subsystem Test Plan 26
Appendix D: Sprint 2 Perception Subsystem Progress Presentation 29
Appendix E: Perception Subsystem Code Snippets 32
Appendix F: Integration Test Plan 33
Appendix G: Technical Document Draft 38
Appendix H: GitHub Excerpts 58

Introduction

The Gallery Greeters project was part of a broader effort to explore interactive, service-based robotics in indoor environments. Designed to be adaptable to any environment, the robot's role was to autonomously greet guests, respond to voice commands, and provide relevant interactions through mobility and perception.

The system was designed to:

- Navigate autonomously using a pre-mapped gallery space
- Interpret voice commands for behaviour switching
- Identify artworks using camera-based perception
- Execute behaviour modes like dancing, patrolling, or returning to the charging station

This project was completed in collaboration with Anika Roth, Andrew Coorey, and Lachlan Scott Rogers. While the overall system was developed as a group, this report focuses on my individual contributions, specifically the development of the Perception subsystem and my role in project management.

My contributions included designing and implementing a ROS2 node for real-time artwork recognition using ORB-based template matching, as well as coordinating system integration, documentation, and final deployment.

The project leveraged tools such as ROS2 Humble, Gazebo, RViz2, Python, OpenCV, and MS Teams for communication. The system was designed and validated over four structured sprints, leading to a deliverable that reflected both real-world robotics constraints and collaborative software engineering.

Sprint Breakdown & Contribution

This sprint established the foundation for the project. Key tasks included team formation, safety and risk planning, defining the system architecture, and outlining evaluation criteria for each subsystem.

Sprint 1 – Foundation & Planning

Sprint Goals

This sprint established the foundation for the project. Key tasks included team formation, safety and risk planning, defining the system architecture, and outlining evaluation criteria for each subsystem.

Group Deliverables

- Signed Group Contract (Appendix A)
- Completed Health and Safety Risk Assessment (Appendix B)
- System Architecture Diagram and data flow between subsystems
- Subsystem role assignments and milestone planning

Individual Contributions

As project manager, I led the drafting of the Group Contract, organized the project both setting up the teams for ease of communication and document storage alongside the setting up of a Git for ease of access and storage between members for the code, and helped define group communication and accountability protocols.
For the Perception subsystem, I began scoping detection strategies, investigating feature-based vs. ML-based methods, and planning how the subsystem would interface with /camera/image_raw and service calls from other nodes.

Client & Coach Feedback

- Contract Feedback: Clarified that “80% success” must be interpreted quantitatively (e.g., 8 out of 10 trials).
- Risk Assessment Feedback: Highlighted that the most likely real-world hazard was other people walking into the robot’s path during testing, particularly when the operator was not in direct line of sight.

Response to Feedback

Based off the feedback received the Risk Assessment was amended to include this.

Sprint 2 – Development & Testing Setup

Sprint Goals

The focus of Sprint 2 was to develop individual subsystems in isolation, prepare test plans, and individually present progress to the coach and client.

Group Deliverables

- Subsystem test plans (see Appendix C – Perception Subsystem Test Plan)
- Working versions of core ROS2 nodes
- Individual progress presentations

Individual Contributions

I worked toward a working version of template matching. Initially working only to match a single decided upon painting. The progress made included:

- loading in the image template
- Performing ORB-based key point matching
- Loading into the Gazebo Environment
- Initial Tests of template matching

I authored the full Perception Subsystem Test Plan, covering five test cases (e.g., single object classification, ML-based detection, multi-object frames). I also delivered a dedicated individual progress presentation (Appendix D), which demonstrated perception functionality, outlined the testing strategy, and previewed integration plans.

Client & Coach Feedback

- *Individual Progress Feedback:* The perception node was progressing well and clearly explained, but I was advised to accelerate development to meet integration deadlines.
- *Test Plan Feedback:* Received full marks with no suggestions for improvement.

Response to Feedback

I restructured my workflow to enable more rapid testing and iteration. This included having an output of confidence, the captured reference frame for matching and the template with the chosen points key points for feature detection, and saved file of last camera frame. I also further expanded the template dataset to prepare for Sprint 3 integration.

Sprint 3 – Integration & Validation

Sprint Goals

The objective for sprint 3 was to further develop my individual subsystem and begin integration of all subsystems together. The aim was to validate integration through both simulated tests and physical trials. The deliverables included a formal Integration Test Plan and a video demonstration showcasing system functionality. The focus was on verifying communication between voice commands, path planning, control, and perception subsystems.

Group Deliverables

- Completed Integration Test Plan (Appendix E) covering five major test cases
- Sprint 3 demonstration video showing robot behaviour in both simulated and hardware settings
- Partial system integration: voice-triggered navigation, artwork identification, and table guidance
- Initial synchronization of ROS 2 nodes for coordinated behaviour switching

Individual Contributions

As the lead developer of the Perception subsystem, I was responsible for implementing and refining the template matching node used to identify artworks. During Sprint 3, I:

- Improved template matching confidence metrics and ORB feature detection parameters
- Validated the perception service's performance under various lighting and angle conditions
- Integrated the */perform_template_matching* service call into the main system behaviour flow
- Managed all debugging associated with incorrect or missed matches during dynamic movement
- Documented results in the Subsystem Test Plan and contributed data to the Integration Test Plan

Client & Coach Feedback

The feedback received highlighted that:

- While individual subsystems were functioning effectively, most were only demonstrated in simulation.
- Only one subsystem (voice command) was shown working with the physical robot.
- The reflection provided in the video lacked depth, and no clear timeline was communicated.

Sprint Demonstration Video

The sprint 3 demonstration video can be found at:
https://youtu.be/Ww0Ku3Afi1Q?feature=shared

Sprint 4 – Documentation & Handover

Sprint Goals

The primary goal for Sprint 4 was to deliver a fully integrated, physical demonstration of our robotic system using the TurtleBot3. This included ensuring all subsystems worked cohesively in a mapped gallery environment, finalizing technical documentation, and handing over a reproducible system ready for external use.

Group Deliverables

- End-to-end integration of all subsystems (Voice, Path Planning, Control, Perception)
- Successful trial run and/or simulated test prior to handover
- Physical demonstration of robot behaviours in the mapped environment
- Finalised Technical Documentation (Appendix G)
- Clean and accessible GitHub repository with setup instructions (Appendix H)

Individual Contributions

I completed and validated the Perception subsystem, ensuring:

- Accurate template matching via ORB feature detection
- Confidence threshold tuning and consistent results in both simulation and real-world trials
- Debug tools and visual logs to verify performance

Additionally, I contributed heavily to environment setup and final testing:

- Assisted with artwork goal coordinates alignment and that they were correctly aligned with the physical map used by the robot
- Supported mapping runs before handover to verify localization and navigation readiness
- Conducted final tests in the demonstration space, verifying that the robot's behaviours were consistent and aligned with expected outputs

Further still:

- Oversaw cross-system coordination and testing
- Polished the technical documentation and assisted in the clarity of instructions for new users
- Contributed to the GitHub README for accessibility, ensuring it could guide new users through setup and deployment without intervention

Pre-Demo Testing and Functionality

In the days leading up to the handover, we conducted multiple tests where the TurtleBot successfully performed all behaviours on hardware, including:

- Navigating to artworks and tables
- Executing patrol and dance routines
- Initiating the perception service upon reaching an artwork
- These tests used voice command in testing mode, where numerical input simulated voice triggers due to the lack of internet access (required for Google Speech-to-Text).

Handover & Demo Day Issues

On the day of the demo, we encountered several unexpected issues, despite prior successful hardware tests:

- The TurtleBot network was extremely slow, leading to launch file and topic communication delays
- Our main launch file failed to fully initialize subsystems as expected
- Most critically, when launching the p_controller on my machine (which we had chosen for the demo), the robot became unresponsive, despite LaserScan and occupancy grid data appearing correctly in RViz — an issue that had not occurred on any machine before, including mine during prior tests

Additional Compounding Factors:

- Andrew's microphone failed to register, blocking real voice input
- Lachlan's Git was out of sync and couldn't fetch the latest version
- Anika's machine was not fully prepared for hardware control

These overlapping issues, all manifesting during the demo window, made it impossible to deliver the intended full physical demonstration, despite confirmed system readiness days prior.

Client & Coach Feedback

“Well done! The documentation is in good shape. However I'd like to see more real-world setup steps, especially with using a real TurtleBot.” The previous was the feedback received from Tony regarding our technical documentation draft.

Response to Feedback

In response to Tony’s feedback regarding clearer real-world setup guidance, I ensured the revision of the technical documentation was undertaken and further worked on enhancing the GitHub README to include:

- Step-by-step installation and build instructions for ROS2 Humble
- Explicit details on required Python packages and external dependencies
- Clear instructions on launching the system, including how to manually start the voice command node
- Notes on setting up the Porcupine API key and adapting file paths for different users
- A fallback setup procedure for running the voice subsystem in testing mode when internet access is unavailable
- Guidance on ensuring the robot is localized correctly using the map, and how to interact with it in simulation or on hardware

These updates were made with the intent to ensure that anyone, without assistance from our team, could clone the repository, install dependencies, and run the project in either simulation or physical deployment.

Perception Subsystem Technical Description

Purpose

The purpose of the perception subsystem is to identify visual features within the environment, with a primary focus on detecting and differentiating paintings affixed to the gallery walls. This is achieved through template matching techniques, allowing the robot to visually confirm the presence and identity of specific artworks, which enables behaviour switching and information display in response to user queries.

Key Interfaces

Source Files

- template_match.py – Implements all perception logic and service handling for template matching
- packages/template_match/resource/ - contains “png” and “jpeg” files required to perform perception matching

Subscribed Topics

- /camera/image_raw – Receives live image stream from the TurtleBot3’s camera for processing

Published Topics

- /interrupt_signal – Publishes a bool message to notify the system of a successful match that may interrupt current behaviour

Service Topics

- /perform_template_matching – process requests to perform template matching using most recent camera image

Published Files (For Debugging Purposes)

- /tmp/latest_camera_frame.jpg – Stores the latest received image frame
- /tmp/match_<label>.jpg – Stores visualised keypoint matches for each template comparison to the last captured frame

Configurable Settings

Settings	Value	Description
Confidence Threshold self.declare_parameter(‘confidence_threshold’, VALUE)	0.65	Minimum confidence required to accept a match result
ORB Features self.orb = cv.ORB_create(nfeatures=VALUE)	1000	Number of ORB features extracted from templates and camera
Top Match Count top_n = VALUE	20	Number of top keypoint matches considered for scoring
Distance Normalization D_max = VALUE	75	Normalization to scale average keypoint match distance

Node description & algorithm logic

The node loads all image templates from the resource/ folder on startup.

- Upon receiving a service request, the system performs ORB feature matching between the current camera image and all available templates.
- For each template, the top 20 matches are evaluated to compute an average distance, which is normalized into a confidence score.
- A debug image is saved showing matched keypoints for each comparison (/tmp/match_<label>.jpg).
- The most recent raw image is also saved to /tmp/latest_camera_frame.jpg for verification and analysis.
- Although the /interrupt_signal publisher is initialized, it is currently commented out in the handle_template_matching function.

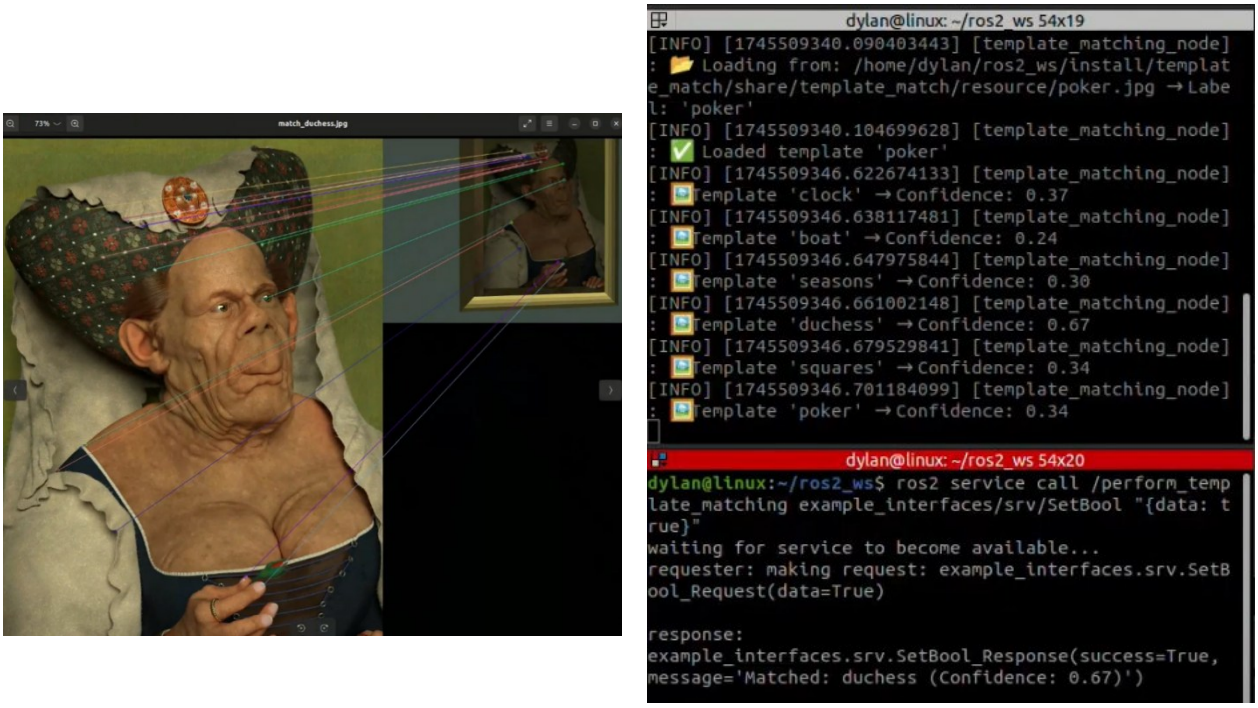
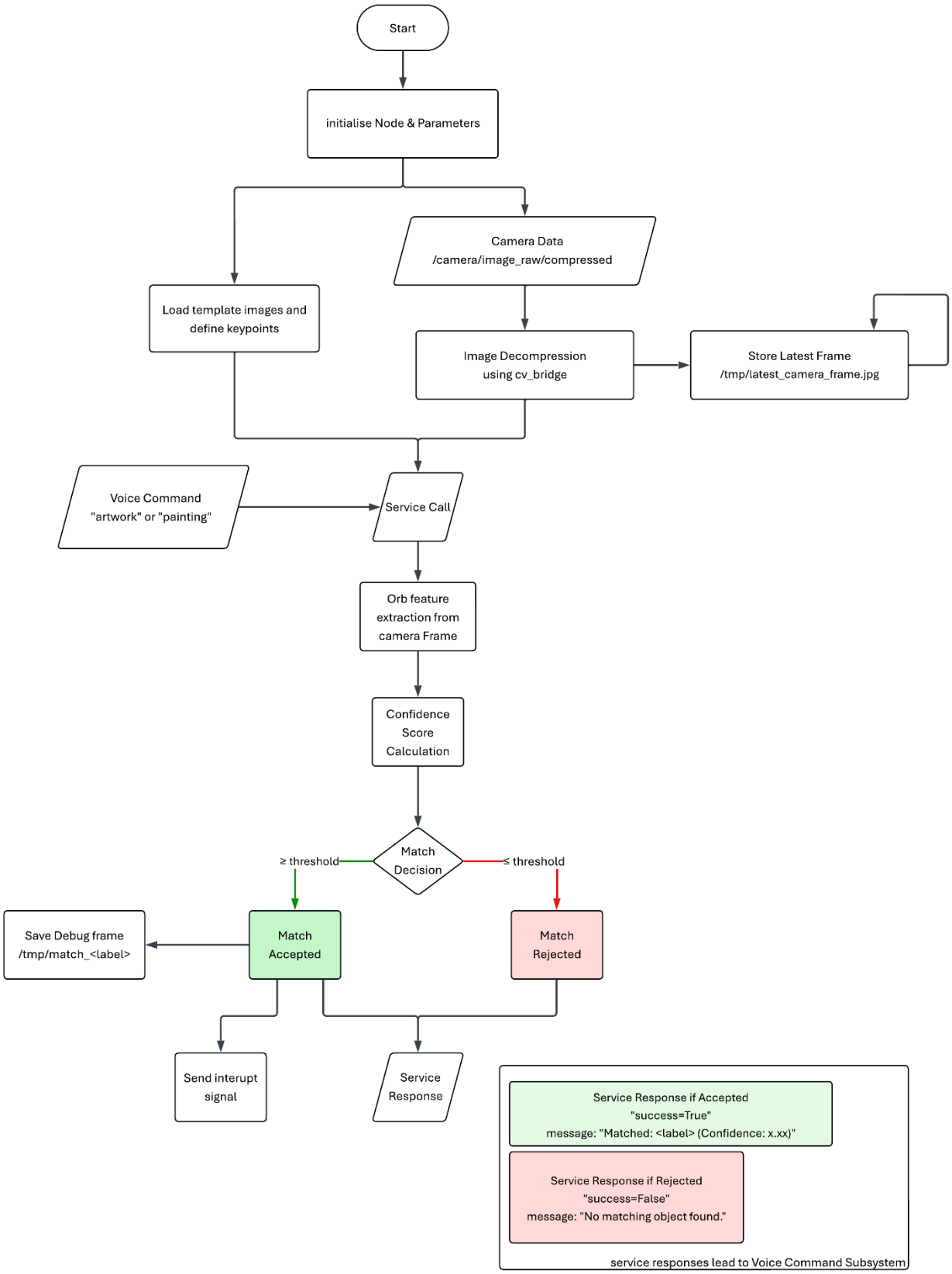


Figure 1 - Results of template matching in Simulation

Perception System Architecture



Team Collaboration

The main lines of communication between all group members apart from face-to-face meetings and time together in the studio was through MS Teams. MS Teams was also the platform we used to collaborate on documentation such as the group contract, integration test plan, as well as other collaborative documents and miscellaneous file storage. GitHub was where we hosted all our code and the elements of our simulated testing environment. The team worked well together to achieve what we have.

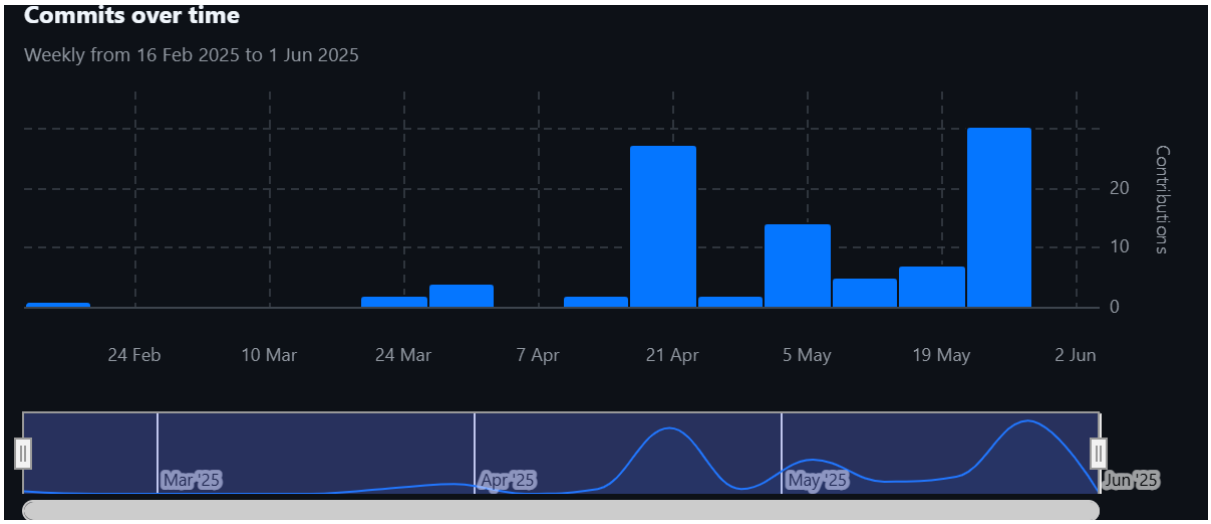


Figure 2 - GitHub Contributions

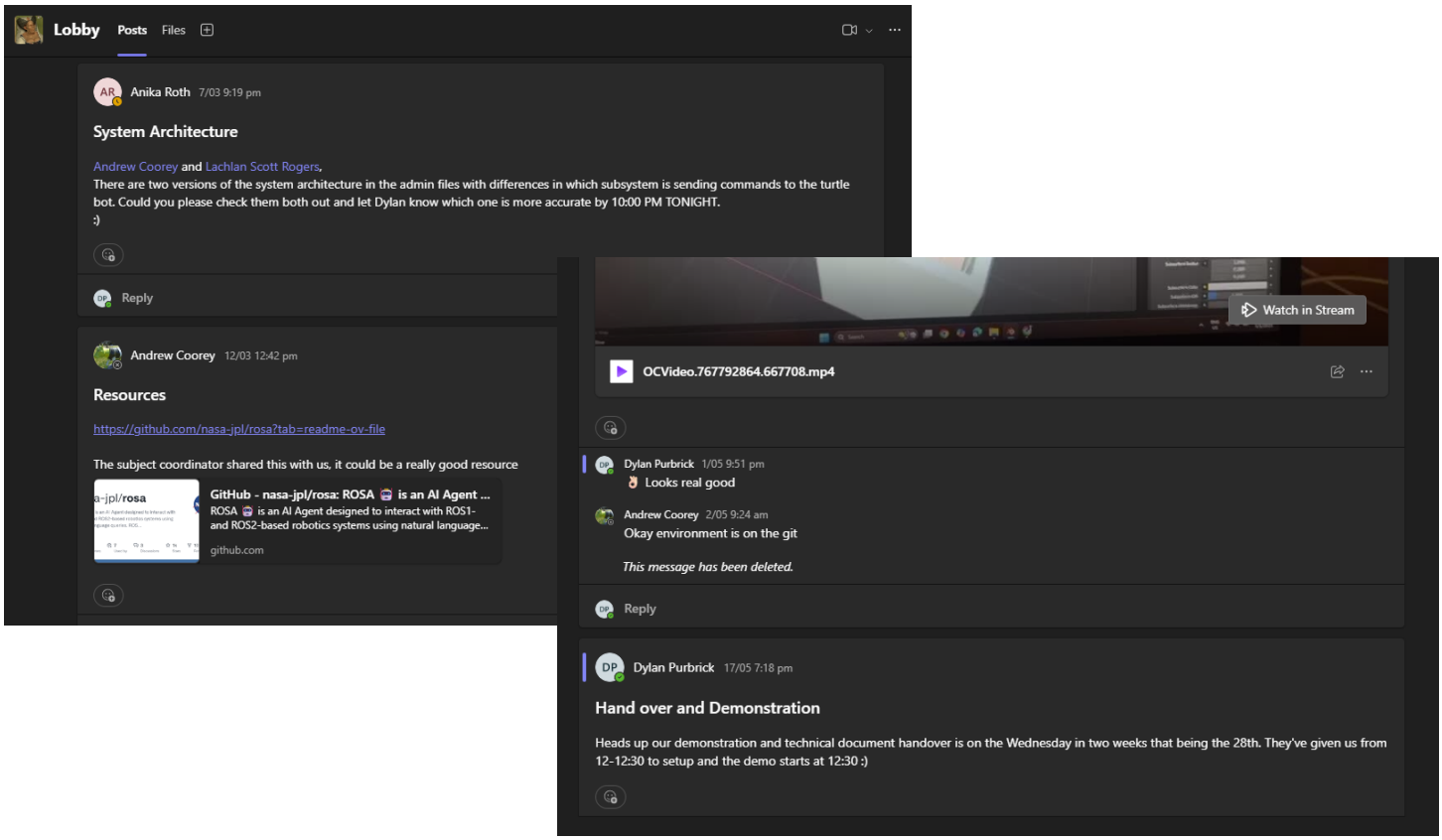


Figure 3 - MS Teams communication

Reflection & Individual Development

This project significantly contributed to my growth as both a mechatronic engineer and a team collaborator. I gained hands on experience in:

Technical Skills

- ROS2 Node development: Designed and implemented a perception node using Python, cv_bridge, and OpenCV
- ORB Feature Matching: Tuned keypoint detectors and confidence scoring to enable real-time artwork identification
- Debugging Multi-node Systems: Tracked down launch errors, handled Rviz visualization mismatches, and improved test logging
- Simulation & Physical Testing: Validated my subsystem in both environments and adapted code to physical constraints like lighting variation

Project Management

- Scheduled sprint tasks and coordinated team progress
- Facilitated documentation updates and final formatting
- Supported hardware setup and testing across multiple machines
- Ensured GitHub repo clarity for external users

Soft Skills

- Enhanced my adaptability, especially under pressure during system demos
- Learned how to lead collaboratively while encouraging contribution from every team member
- Developed concise technical writing through system documentation and GitHub README formatting

Key Lessons

- Test early and often on the physical system — even if everything works in simulation, hardware introduces subtle failures.
- Build for clarity and reuse — well-commented code and modular services are invaluable when testing under time constraints.
- Stay proactive with contingency planning — our testing mode for voice input was essential when internet was unavailable

Conclusion

The Gallery Greeters project successfully integrated perception, planning, control, and user interaction into a cohesive, autonomous robotic system. Over four structured sprints, we developed a ROS2-based platform that demonstrated strong subsystem coordination and reliable functionality. I particularly enjoyed the time spent in the lab spaces, collaborating on design improvements and refining the system through hands-on testing and iteration.

As the developer of the Perception subsystem and overall project manager, I was directly responsible for implementing the visual recognition functionality and ensuring the system could be reproduced by others through clear documentation and a structured GitHub repository. Although unforeseen technical issues prevented a full physical demonstration on handover day, the system was successfully demonstrated in simulation, with all core features validated in prior hardware tests.

This project aligns closely with my long-term goals in robotic vision and autonomous systems. It has provided me with valuable technical, collaborative, and problem-solving experience that will support future research, industry roles, and advanced study in robotics and artificial intelligence.

Potential Future Development:

- Replacing template matching with deep learning-based recognition
- Achieving full voice integration with offline support
- Expanding the perception subsystem through machine learning methods
    - This being the Distinction and High Distinction levels
- Enabling detection of broader object types beyond artworks

The Gallery Greeters robot stands as a deployable prototype, and this project has been a defining milestone in my development as a mechatronics engineer.

Appendix

Appendix A: Group Contract

Gallery Greeters Project Contract

Client: *Louis Fernandez*

Coach: *Isira Wijegunawardana*

Team Members

Name:	Email:
Andrew Coorey	andrew.g.coorey@student.uts.edu.au
Anika Roth	anika.roth@student.uts.edu.au
Dylan Purbrick	dylan.m.purbrick-1@student.uts.edu.au
Lachlan Scott Rogers	lachlan.w.scottrogers@student.uts.edu.au

Name	Skills/Strengths	Group Skills to Work On	Stressors
Andrew Coorey	• Programming • Working collaboratively	• Being assertive and decisive • Asking for feedback	• Missing deadlines • Lack of communication
Anika Roth	• Programming	• Meeting stakeholder's needs	• Unbalanced work loads
Dylan Purbrick	• Communication • Research	• Time management • Programming	• Having no accountability
Lachlan Scott Rogers	• Programming	• Time management • Communication	• Conflicting deadlines

Overview

An automatic robotic solution is being developed for the Tom Ugly's Art Gallery's opening art exhibition. The solution will be part of a three-stage system, the Gallery Greeters project will handle the hosting role of the gallery. The behaviour of the robot will be to lead guests to available tables, recognise vocal inputs and requests from guests including taking drink orders and providing further information about the artworks on display. The host robot will need to dynamically avoid guests and other objects within the environment when planning paths to the tables.

Proposed solutions:

1. Autonomous navigation system - The robot will utilise sensors, path planning algorithms and a control method to avoid obstacles while efficiently guiding visitors to their tables.
2. Voice recognition and interaction - The system will be equipped with a microphone and speaker to understand and respond to vocal requests (e.g. “take me to a table”, “Can I get a drink”, “Tell me about this artwork”).
3. Integration with artwork database – The system will have access to a digital repository with detailed information on the showcased artworks, allowing the robot to provide insightful information to guests when asked.
4. Visual perception of environment – The system will be able to identify unique objects within the environment. This will be used for identifying artworks, obstacles, available tables and features within artworks.

Key Challenges:

1. Path planning– The system will need to navigate the environment finding the most efficient path to its destination.
2. Obstacle avoidance – The system will need to dynamically avoid obstacles interrupting the planned path to create a new path around the obstacle to move safely to its destination.
3. Accurate Voice Recognition – The system will need to identify guest voices; this will involve filtering out background noise and recognising requests.
4. Visual servoing – Being able to identify and distinguish between objects (e.g. paintings, humans, tables etc.)

Aims

Goals:

- To both simulate and physically run a robot that will:
    - lead guests to available tables
    - take drink orders
    - provide guests with information on artworks around the gallery
- Program the robot to react to voice commands (e.g. “take me to a table”, “Can I get a drink”, “Tell me about this artwork”)
- Program the robot to be able to avoid obstacles as it navigates around the environment
- Program the robot to be able to identify objects within the environment as well as distinguish between different artworks

Milestones:

Milestone 1: Develop group contract, subsystems, responsibilities and evaluation criteria

Milestone 2: Develop world environment, mapping and basic connection test with TurtleBot3

Milestone 3: Establish stable connection with hardware including the TurtleBot3 and auxiliary hardware

Milestone 4: Turtlebot3 can complete autonomous path planning

Milestone 5: Control method for obstacle avoidance is developed

Milestone 6: Implementation of object detection and identification

Milestone 7: Consistent Voice Command Recognition is achieved

Milestone 8: Full System Integration is completed

Deliverables:

- A path planning algorithm that can autonomously go from point A to point B
- A control method for object avoidance that dynamically adjusts the planned path to avoid obstacles
- A voice recognition algorithm that successfully interacts with patrons and executes requests
- An implementation of a DL/ML model for use of object detection and/or identification
- Configuration of the physical robot to suit the needs of the project
- Simulated environment of the art gallery for the robot to navigate and test in
- Reproduction of the simulated environment for the physical robot to be tested in

Resources

Describe the resources required to complete the project. This includes: the robotics hardware & software available in the Lab, additional hardware & software you might need to source yourself, fabrication access (3D printing, laser cutting, etc.).

Hardware	*Software*	*Other*
• TutrleBot3 (camera, lidar and external connectivity access) • Microphone • Digital Screen	• Linux • ROS2 • Gazebo • VSCode	• GitHub • Microsoft Suite (Teams, Outlook) • Data Points (Images for object detection) • Fabrication Access (3D printing and laser cutting)

Subsystems and Responsibilities

Subsystem	Description	Lead by	Supported by
Voice Commands	Call and response of robot to person. Call – Audio command prompts Response – Visual or Audio output of prompts or information	Anika	Dylan
Path Planning	Create a path planning method that will create paths for the TurtleBot to take within a known environment	Lachlan	Anika
Control	Create a method of controlled obstacle avoidance around objects within the environment	Andrew	Lachlan
Perception	Object detection and identification. Distinguishing unique objects and similar objects from each other	Dylan	Andrew

Evaluation

Subsystem 1 – *Voice Command*	
P	Understands 5 out of 10 commands consistently (80%)
C	Understands 10 out of 10 Commands consistently (80%)
D	Implement a ML model for 10 voice commands to be recognised in less controlled environments 80% of the time
HD	Create own DL model for 10 voice commands to be recognised in less controlled environments 85% of the time
Extension	Incorporate LLM to have conversational feedback
Subsystem 2 – *Path planning*	
P	TurtleBot moving in environment with operator control (real and simulated)
C	Basic autonomous pathfinding using existing library
D	Create a basic path planning method to obtain a path between two nodes
HD	Create a more advanced path planning method that optimises the path based on acceleration or distance or trajectory
Extension	Implement a probabilistic path planning approach
Subsystem 3 – *Control*	
P	Create a P controller to reach a desired location
C	Perform static obstacle avoidance (potential field approach)
D	Create a PD controller with obstacle avoidance
HD	Utilise optimisation method for obstacle avoidance (e.g. QP controllers)
Extension	Avoid dynamically moving objects, this can be humans or other robots within the environment
Subsystem 4 – Perception	
P	Differentiate between a set of 5/10 objects 80% of the time
C	Differentiate between a set of 10/10 objects 80% of the time
D	Implement a ML model to differentiate between similar objects 80%
HD	Implement a CNN model to differentiate between similar objects 80%
Extension	Use an object detection model to differentiate different objects in the same image

Team Processes

Communication:

The team will communicate through numerous channels including MS Teams, MS Outlook and in-person meetings. The frequency of communications will occur on average every 3 days, with a maximum response time of 5 days. Team meetings will also be held in/or before every lab and studio class to ensure that all team members are briefed on recent project developments and the agenda for the subsequent class.

Decision-Making:

Decisions that will affect the whole project or impact on multiple team member's works will be decided through group consultation. Such consultation may only require the attendance of specific team members impacted by the decision, or by the entire team. The consultation medium may take the form of a meeting (in-person or online), email, chat message, chat poll or another suitable mode. The chosen resolution will be the one that the majority of impacted members agree upon. Individual team members may make their own non-critical decisions if they lie within their scope of work and are negligible to other team members' work.

Conflict Resolution:

All conflicts must first attempt to reach a resolution within the project team, either between specific individuals, or by the whole project team. Should this fail or lead to an impasse, the issue may be escalated to the class tutor and/or subject coordinator for intervention.

Commitments:

It is expected that all team members participate and contribute an equal amount to the delivery of the project in both technical, and non-technical areas. The entire project team will determine the satisfactory standard of specific deliverables to ensure consistency and cohesion between different aspects of the project. Should any member/s consistently fail to meet this standard of quality or participation, they may be referred to the class tutor and/or subject coordinator.

Diversity:

Diverse learning and working styles will be accommodated to optimise the performance of all team members. Within reason, generous deadlines will be offered for project deliverables to ensure that all team members can proceed on their own schedule. Specific expectations for working conditions can be established as a team or individuals see fit.

Other Procedures:

It is the responsibility of all team members to ensure that all project developments, blockages or failures are recorded and made known to other members both in-person and/or by posting to teams a description of the issue.

Signatures:

Member 1: **Andrew Coorey** Date: **07/03/2025**

Member 2: **Anika Roth** Date: **07/03/2025**

Member ***3:*** **Dylan Purbrick** Date: **07/03/2025**

Member 4: **Lachlan Scott Rogers** Date: **07/03/2025**

Appendix B: Risk Assessment

UTS HEALTH, SAFETY & WELLBEING

GENERAL RISK ASSESSMENT

Activity name	Robotics Studio 2 – Gallery Greeters	**Faculty**	FEIT	**School / Centre / Department**	MME
Activity description	Testing and demonstration of TurtleBot 3 to perform navigation of a set area and avoidance of obstacles within.				
Date of assessment		**Version No.**		**Next review date**	
Form completed by/Assessor:		Staff / ~~***Student***~~ / Other	**Supervisor**		
Location(s) of activity, e.g. CB01.01	CB11.10.403	**Lab Manager/Technical Services (or equivalent) of activity location (if relevant):**	Felipe Gutierrez	**Planned activity date(s):**	
Persons at risk	Workers / ~~***Students***~~ / Visitors / Contractors / Public / Other	**Persons consulted** (consider anyone with access to or affected by the activity)	Andrew Coorey, Anika Roth, Dylan Purbrick, Lachlan Scott Rogers		
List any Reference legislation, standards, codes of practice, manufacturer's guidance etc used to help identify hazards and control measures relevant to this activity *Refer to the H&S Policy, Codes of Practice, Australian Standards*					

TASK List and describe hazardous task/activity/process/step/equipment	**ASSOCIATED HAZARD(S)**	**INHERENT HARM** Harm that could occur from these hazards if controls fail or are not in place.	**EXISTING CONTROL MEASURES** Control measures currently in place to minimise risk	**RISK LEVEL** (H,M,L)	**PROPOSED CONTROL MEASURES** Additional control measures needed to reduce risk further	**TARGET DATE** To implement proposed controls	**RESIDUAL RISK LEVEL** (H,M,L)
Electrical cables mishandled	Electric Shock Short Circuit	Electrical Fires	• UTS standard cable management practices	L	• Inspecting Cables before operation for signs of wear/damage	11/03/25	L
Leaving the robot unattended	Someone could unknowingly run the program without observers aware	Robot could malfunction and result in physical harm to the operator and observers	• Having both a lab supervisor and assessor present to monitor the operation of the robot	M	• Ensure when a supervisor/assessor is not present that the robot is turned off completely. • Keep observers at a distance from operation area	11/03/25	L
Operator getting caught by the robot	Crushing/Trapping	Operator/observer injury	• Emergency stop button	H	• Having the emergency button positioned near the supervisor for quick action in an emergency	11/03/25	L
Incorrect installation/manual handling	Unexpected malfunction of robot	Physical harm	• UTS periodic inspection and supervision while assessor is operating robot	M	• Train operators with the correct installation and handling procedures	11/03/25	L
Operating robot when tired or distracted	Failure in manual handling and operation	Robot could malfunction and result in physical harm to operator and observers due to negligence	• Lab supervisor present to assess operator/assessors' capability of the robot	M	• Elimination of all potential distractions	11/03/25	L
Use of LiPo batteries	Potential fire and/or explosion	Physical harm Damage to room/building due to fire	• Supervised/controlled use of batteries • Implementation of fuses	M	• Checking over the battery before use • Not leaving the battery plugged in while unattended	11/03/25	L

TASK List and describe hazardous task/activity/process/step/equipment	**ASSOCIATED HAZARD(S)**	**INHERENT HARM** Harm that could occur from these hazards if controls fail or are not in place.	**EXISTING CONTROL MEASURES** Control measures currently in place to minimise risk	**RISK LEVEL** (H,M,L)	**PROPOSED CONTROL MEASURES** Additional control measures needed to reduce risk further	**TARGET DATE** To implement proposed controls	**RESIDUAL RISK LEVEL** (H,M,L)
The use of a shared lab space by other people	Potential interference with the robot's operation, accidental collisions, obstruction of pathways	Physical harm and injury Damage to robot Disruption of testing procedures	• Clear marking of the robot's operating area • Verbal communication with other lab users before running tests	M	• Implement signage to indicate active testing	20/03/25	L

Emergency preparation and response					
EMERGENCY List and describe foreseeable potential emergency situations	**INHERENT HARM** Harm that could occur from these hazards if controls fail or are not in place.	**EXISTING CONTROL MEASURES** Control measures currently in place to minimise risk	**PROPOSED CONTROL MEASURES** Additional control measures needed to reduce risk further	**TARGET DATE** To implement proposed controls	**RESIDUAL RISK LEVEL** (H,M,L)
Fire within the building	Physical harm, death, trauma	Fire alarms, sprinkler system, fire wardens and extinguishers	Being aware of assembly points and safety procedures	11/03/25	L
Flooding of building	Physical harm, death, trauma	Assembly areas	Being aware of assembly points and safety procedures	11/03/25	L
Injuries from robot	Physical harm	Lab supervision	Being aware of the robots' position and operational status	11/03/25	L

Sign-off and Approval						
Form/Assessor Author:		Reasonably practicable control measures identified and implemented	Signature		Date	
Responsible supervisor's* Name:		Satisfied that control measures will reduce risk to an acceptable level	Signature		Date	

*Responsible supervisor is the person with control/authority over the activity

Acknowledgement of Understanding

Persons performing the activity/tasks sign that they have read and understood the risk assessment.
Note: For activities which are low risk or include a large group of people (e.g. open days, BBQ's, student classes etc), only the persons undertaking the key activities should sign below. For all others involved, the information can be covered by other methods (for example a safety briefing, induction, and/or safety information sheet).

Student / Staff name	ID	Date	Signature	Remarks
Andrew Coorey	13940085	19/03/25		
Anika Roth	24456156	19/03/25		
Dylan Purbrick	24945343	19/03/25		
Lachlan Scott Rogers	24537235	19/03/25		

Guidance notes for completing the risk assessment

TASK

Briefly describe hazardous tasks involved in this work activity – For example, Operating, Handling, Using … (Include names) of hazardous equipment, substances or materials used, and any quantities and concentrations of substance(s) or reaction products.

ASSOCIATED HAZARDS

Manual Handling – moving objects, repetitive movements, lifting awkwardly, lifting heavy objects
Work Environment – extremes in temperature, work at height, explosive atmosphere, slippery surfaces/trip hazards, work load, work alone, work after hours, confined spaces, infrastructure
People – potentially violent or volatile clients/interviewees, harassment, bullying, victimisation, poor culture
Environmental – emissions to atmosphere, discharge to soil and water bodies (including stormwater run-off), nuisance noise & odour, poor ventilation/air quality
Plant & Equipment – noise, vibration, dust, moving parts (crushing, friction, stab, cut, shear), pressure vessels, lifts/hoists/cranes, sharps, maintenance, design/assembly, AEV/Drone, hot work
Electrical – plug-in equipment used in 'hostile' work environment, exposed conductors, high voltage equipment
Chemical – hazardous substances, dangerous goods, fumes, dust, compressed gas, hazardous waste
Biological – exposure to bodily fluids/infectious materials, pathogenic microorganisms (bacteria, viruses, parasites, fungi), security sensitive biological agents, sharps/needles, animal bites and scratches, allergies to animal bedding, dander and fluids
GMOs – dealings with genetically modified organisms
Cytotoxins – carcinogens, mutagens or teratogens
Radiation (ionizing) – Ionizing radiation source such as radioactive substance or radionuclide, or irradiating apparatus
Radiation (non-ionizing) – including lasers, microwaves or UV light

INHERENT HARM

Provide details of the harm that could be caused to people or the environment if something goes wrong.
For example: inhalation of fumes, laceration, injury to back, infection, burns to skin or eyes.
Think about what could happen if controls fail or are not in place.

EXISTING CONTROL MEASURES

This is existing measures in place to reduce risk to an acceptable level. Apply the "Hierarchy of Controls", listed below, when deciding the best control measure to apply. Control types closer to the top of the list are preferable.

1. ELIMINATE THE HAZARD. For example, work from the ground with a long-handled tool instead of a ladder thus eliminating work at height.
2. SUBSITUTE THE HAZARD. For example, use a less dangerous piece of equipment or chemical.
3. ISOLATE THE HAZARD FROM PEOPLE. For example, move a noisy equipment into a room that is not accessed when it is in operation.
4. USE ENGINEERING CONTROLS. For example, use a fume cupboard for chemicals, use a guard for rotating parts.
5. USE ADMINISTRATIVE CONTROLS. For example, change work practices, provide training, use signage, develop a safe work method statement.
6. USE PERSONAL PROTECTIVE EQUIPMENT (PPE). For example, respirator, hearing protection, gloves. Training and information is required for the use of PPE.

PROPOSED CONTROL MEASURES

List control measures not currently in place but you plan to put in place before the activity starts.

RISK LEVEL (High / Medium / Low)

The level of risk can be determined by combining consequence and likelihood using the risk matrix below. Residual risk is the level with all control measures in place (existing and proposed). It should be reduced to a level acceptable by management.

CONSEQUENCE OF HARM - This is how bad it will be if something does go wrong e.g. the number of people that could be harmed, the severity of injury.

LIKELIHOOD OF HARM - Chance of harm occurring is affected by the duration of the activity and its frequency; the number of people doing the activity and the level of exposure to the hazard.

For more information on risk determination refer to the [UTS Risk Management Procedure]

			Health & Safety Risk				
Likelihood	The risk is expected to occur in most circumstances	Almost certain	Moderate	High	High	Critical	Critical
	The risk will probably occur in most circumstances	Likely	Moderate	Moderate	High	High	Critical
	The risk should occur at some time	Possible	Low	Moderate	Moderate	High	High
	The risk could occur at some time	Unlikely	Low	Low	Moderate	Moderate	High
	The risk may only occur in exceptional circumstances	Rare	Low	Low	Low	Moderate	Moderate
			Insignificant	Minor	Moderate	Major	Catastrophic
			Non-injury incident or first aid treatable injury only	Illness or injury that requires medical attention leading to 0 to 10 days off work but does not permanently impact health or wellbeing of an individual	Injury or ill health requiring medical attention leading to over 10 and up to 20 days off work but does not permanently impact health or wellbeing of an individual	Injury or ill health requiring hospital admission leading to over 20 days off work but does not permanently impact health or wellbeing of an individual	Single or multiple deaths or a serious injury that permanently impacts health and wellbeing of an individual
			Consequence				

Appendix C: Perception Subsystem Test Plan

Perception Subsystem Test Plan

Test 1: Rudimentary Object Classification

Requirement Addressed

The system must detect a single object of the set (tables, artworks, humans, etc.) with 80% accuracy.

Assumptions

- The TurtleBot3's camera and LiDAR are functional.
- The environment has consistent lighting.

Test Procedure

1. Place object in view of the robot's camera.
2. Run the perception algorithm and record detected objects.
3. Compare detection results with ground truth labels.

Pass/Fail Criteria

The test passes if at least 80% (8 of 10 times tested) of the time a single object is correctly identified.

Additional Resources Needed

Pre-defined dataset of gallery objects.

Test 2: Object Classification

Requirement Addressed

The system must classify from a set of minimum 10 objects at least 10 objects (e.g., artworks, furniture) with 80% accuracy.

Assumptions

- The TurtleBot3's camera and LiDAR are functional.
- The environment has consistent lighting.

Test Procedure

1. Place 10 different objects in the robot's field of view.
2. Run the perception algorithm and record detected objects.
3. Compare detection results with ground truth labels.

Pass/Fail Criteria

The test passes if at least 10 objects of the set are correctly classified with 80% accuracy (8 of 10 times tested).

Additional Resources Needed

Pre-defined dataset of gallery objects.

Test 3: Artwork Recognition using ML/CNN

Requirement Addressed

The perception system must correctly classify at least one artwork from a predefined dataset with 80% accuracy using a trained ML/CNN model.

Assumptions

- The robot has access to an artwork database.
- The robot's camera can capture clear images under consistent lighting conditions.
- The perception system uses a trained ML/CNN model for classification.

Test Procedure

1. Display artworks from the dataset in the robot's field of view.
2. Run the ML/CNN based object detection algorithm.
3. Compare recognition results with database labels.

Pass/Fail Criteria

The test passes if the system correctly identifies one of the artworks from the set with 80% accuracy (8 of 10 trials).

Additional Resources Needed

Dataset of artworks with labels.
Trained ML/CNN model for artwork classification

Test 4: Similar Object Differentiation

Requirement Addressed

The perception system must accurately differentiate between visually similar objects (e.g., different paintings, tables of different colours) with at least 80% accuracy using a trained ML/CNN model.

Assumptions

- The perception system uses a trained ML/CNN model for classification.
- The camera has sufficient resolution to capture detailed object features.
- The environment has consistent lighting.

Test Procedure

1. Display pairs of visually similar artworks or objects
2. Run the ML/CNN-based classification algorithm.
3. Measure how accurately the system differentiates them

Pass/Fail Criteria

The test passes if at least 80% of similar objects (8 out of 10 trials) are correctly classified.

Additional Resources Needed

Labelled dataset with visually similar objects.
Pre-trained ML/CNN model for fine-grained object differentiation.

Test 5: Object Detection in the same Frame

Requirement Addressed

The system must correctly identify objects when multiple objects are placed close together using an object detection model (**ML** or **CNN**).

Assumptions

- The perception system uses a trained object detection model (ML or CNN) for classification
- The camera resolution and processing power are sufficient
- The environment has consistent lighting.

Test Procedure

1. Place multiple objects in a cluttered setting
2. Run the ML/CNN-based object detection algorithm
3. Compare detection results with ground truth labels

Pass/Fail Criteria

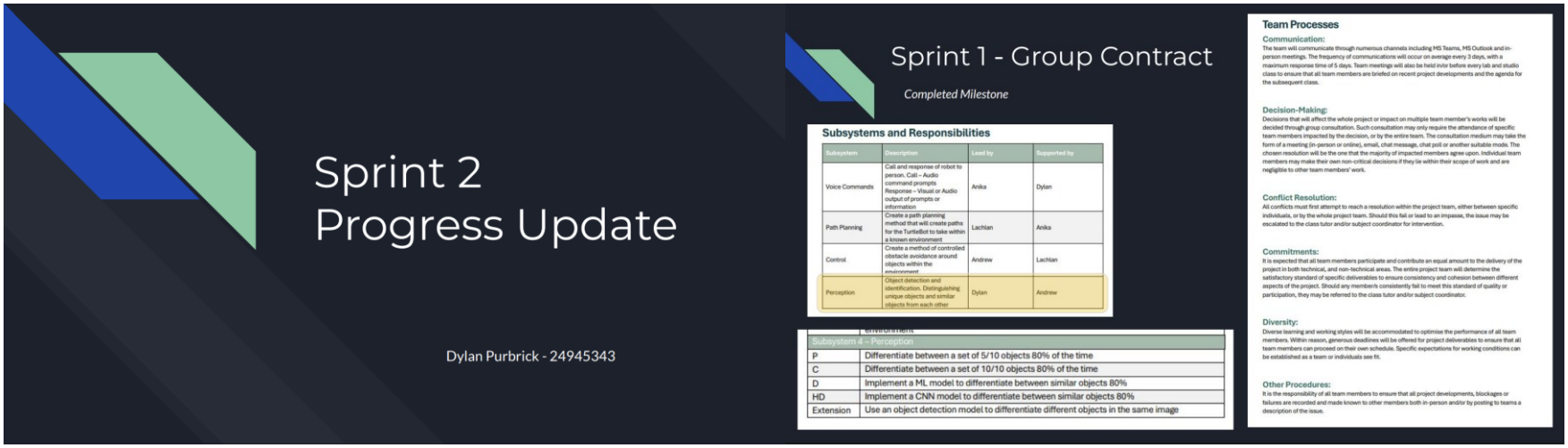
The test passes if at least 80% (8 of 10 trials) of objects are correctly identified.

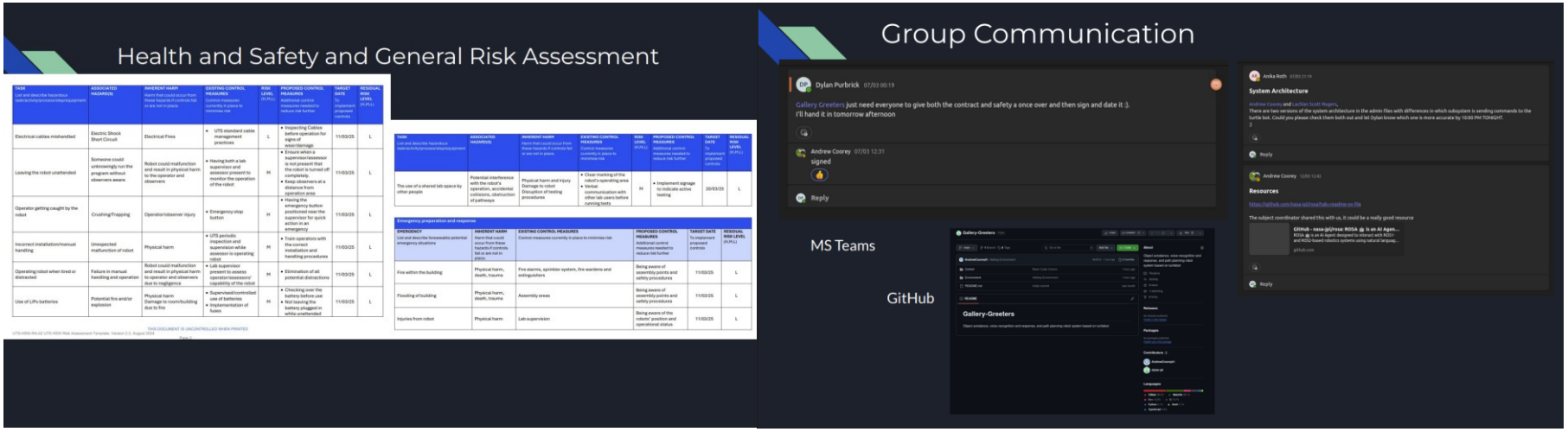
Additional Resources Needed

A labelled dataset with objects placed close together and a pre-trained ML/CNN model.

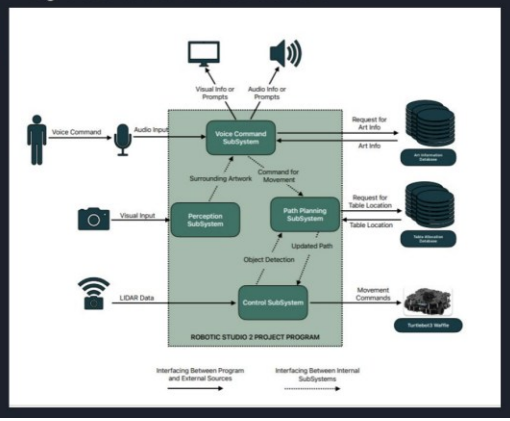
Appendix D: Sprint 2 Perception Subsystem Progress Presentation

The following presentation for sprint 2 can be found here in full.

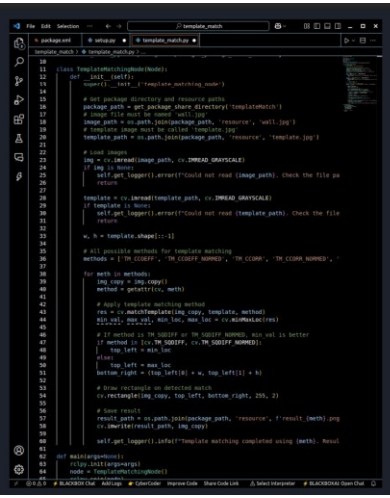




System Architecture



Perception Test Plan

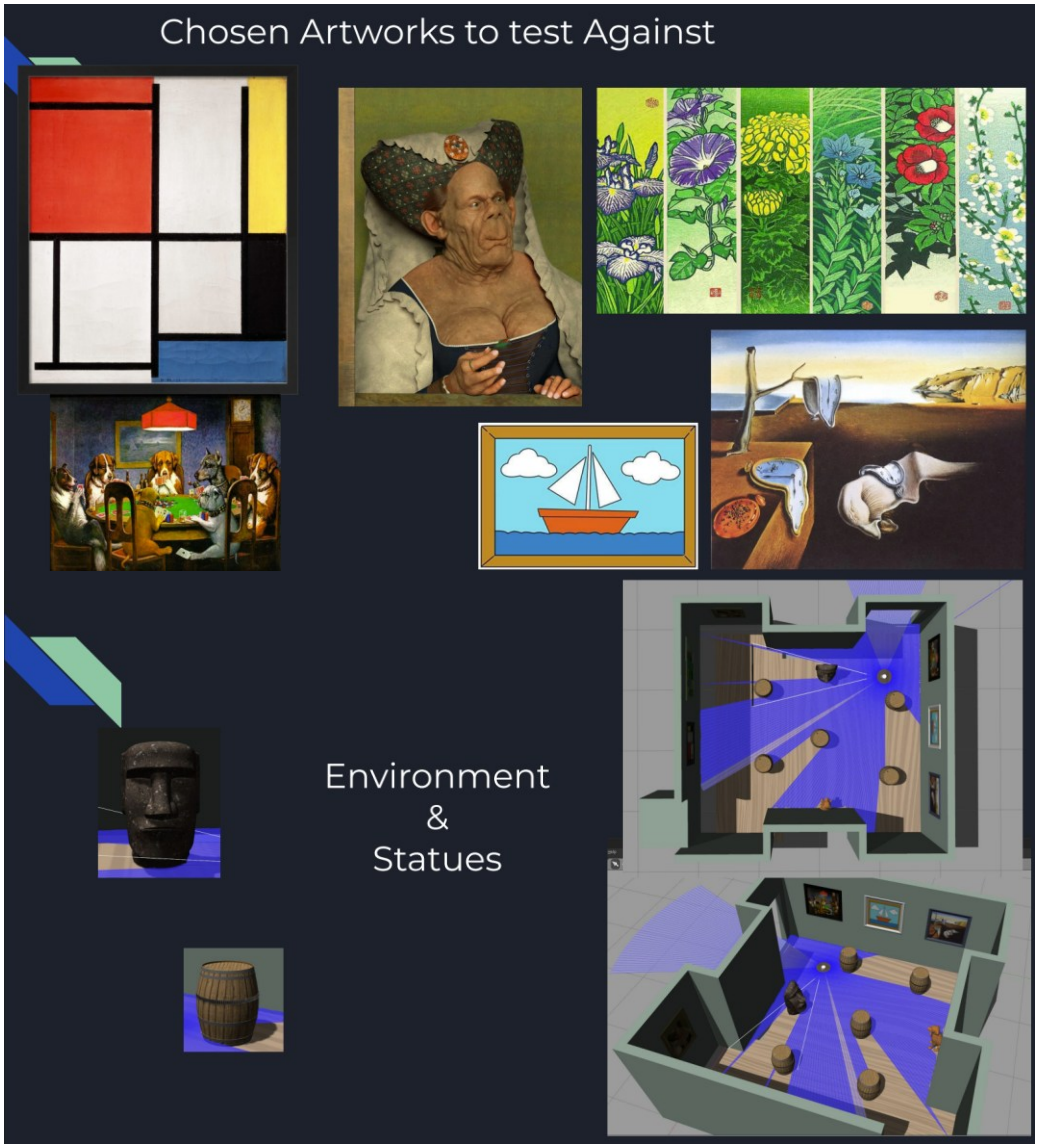


Code Excerpt

Issues

setting linux up and the ros2 work space

File Pathing Issues


Chosen Artworks to test Against
Environment
&
Statues

Appendix E: Perception Subsystem Code Snippets

```python
def load_templates(directory, orb, logger=None):
    templates = {}
    for file in os.listdir(directory):
        if file.lower().endswith(('.jpeg', '.jpg', '.png')):
            path = os.path.join(directory, file)
            image = cv.imread(path, cv.IMREAD_COLOR)
            label = os.path.splitext(file)[0].lower()

            if logger:
                logger.info(f"📂 Loading from: {path} → Label: '{label}'")

            if image is not None:
                kp, des = orb.detectAndCompute(image, None)
                templates[label] = {
                    'image': image,
                    'keypoints': kp,
                    'descriptors': des
                }
            else:
                if logger:
                    logger.warn(f"⚠️ Failed to load template: {file}")
    return templates

```

Template Loading code

```python
def match_templates(frame, templates, orb, logger=None, confidence_threshold=0.65):
    kp2, des2 = orb.detectAndCompute(frame, None)
    bf = cv.BFMatcher(cv.NORM_HAMMING, crossCheck=True)

    best_label = None
    best_confidence = 0.0
    best_matches = []

    D_max = 75
    top_n = 20

    for label, data in templates.items():
        des1 = data['descriptors']
        if des1 is None or des2 is None:
            continue

        matches = bf.match(des1, des2)
        matches = sorted(matches, key=lambda x: x.distance)

        if len(matches) < top_n:
            continue

        avg_dist = np.mean([m.distance for m in matches[:top_n]])
        confidence = max(0.0, 1.0 - avg_dist / D_max)
        if logger:
            logger.info(f"🔍 Template '{label}' → Confidence: {confidence:.2f}")
        else:
            print(f"🔍 Template '{label}' → Confidence: {confidence:.2f}")

        img_matches = cv.drawMatches(data['image'], data['keypoints'], frame, kp2, matches[:top_n], None, flags=2)
        cv.imwrite(f"/tmp/match_{label}.jpg", img_matches)

        if confidence > best_confidence:
            best_confidence = confidence
            best_label = label
            best_matches = matches[:top_n]

    if best_confidence > confidence_threshold:
        return best_label, best_confidence, best_matches
    else:
        return None, 0.0, []
```

Template Matching

The main script of code for the Perception node can be found within the template_match folder of the repository here. Other elements relating to the perception subsystem can be found within the template_match subfolder within packages here, this includes elements of the code and related items such as the tested images, launch file that of which was used to simplify testing of the subsystem in isolation and was lightly adapted for full integration.

Appendix F: Integration Test Plan

Integration Test Plan

Test 1: Artwork Request & Response

Requirements:

The robot must respond to a spoken request to identify an artwork and return the correct label using the perception subsystem.

Assumptions:

- Robot is stationary and positioned in front of a visible artwork.
- The voice recognition system can process a basic command.
- The perception system has access to preloaded artwork templates.
- The camera is unobstructed and functioning.

Procedure:

1. Place the robot facing an artwork present in its template dataset.
2. Speak the command: "Tell me about this art?"
3. The voice subsystem interprets the command and triggers the template matching node.
4. The perception system analyses the camera frame and attempts to identify the artwork.
5. The robot verbally or textually returns the matched artwork label and confidence score.

Pass or Fail Criteria:

Criteria	*Passed*
Request for artwork name is successfully sent and received at least 5 times in a row	
Send the name of the correct artwork 4 out of 5 times with 70%+ confidence	
Voice Command receives artwork name and does correct outputs 4 out of 5 times	

Extra Resources Necessary:

- Microphone input
- Active ROS 2 nodes for both voice command and template matching.
- Template images loaded in /resource.
- A known artwork visible to the camera.

Test 2: Path Planning to Movement

Requirements:

- Path planning subsystem successfully generates a valid path, which is published to a topic for the control subsystem to publish velocity commands.

Assumptions:

- The environment has been previously mapped using SLAM.
- The current map accurately represents the layout and obstacle positions.
- The robot is localized and receiving accurate odometry data.
- All relevant ROS 2 nodes for path planning and control are active.

Procedure:

1. Place the robot at the entrance/start location.
2. Load the previously mapped environment.
3. Launch all required nodes for path planning and control.
4. Set the robot's initial pose within the environment.
5. Specify a goal location within the mapped environment.
6. Observe that a valid path is generated from the current pose to the goal by the path planning node.
7. The computed path is published to the control subsystem.
8. The control subsystem receives the path and generates velocity commands.
9. Record whether the robot reaches the goal within specified tolerance and without running into obstacles.

Pass or Fail Criteria:

Criteria	*Passed*
Path received by Control Subsystem is valid, avoids known obstacles, is accessible >80% of the time	
Robot successfully converts the given path to velocity commands and automatically moves per the given path 80% of the time	
Robot successfully reaches the end goal of the path within 0.1m and doesn't collide with known or unknown obstacles >80% of the time	

Extra Resources Necessary:

- TurtleBot3 robot (physical or simulated)
- Pre-generated environment map
- Active ROS 2 nodes for path planning and control subsystems

Test 3: Navigate & Identify Artwork

Requirements:

The robot must autonomously move to the position of a known artwork using its path planning and control subsystems and then use perception to identify and report the artwork.

Assumptions:

- A known artwork is located at a mapped position in the environment.
- The robot's camera has a clear view once in front of the artwork.
- The artwork template exists in the perception dataset.
- All relevant ROS 2 nodes are active (path planner, controller, perception).

Procedure:

1. Set a navigation goal directly in front of a known artwork location.
2. Trigger the navigation action using the internal controller or planning node.
3. Robot autonomously navigates to the target location, avoiding static obstacles if necessary.
4. Upon arrival, the robot halts and activates the perception system.
5. The perception node attempts to identify the artwork using template matching.
6. Detected label and confidence score are logged or announced.

Pass or Fail Criteria:

Criteria	*Passed*
The robot reaches the predefined artwork location and triggers perception subsystem successfully 80%	

Extra Resources Necessary:

- Predefined map with accurate artwork locations
- Template images for known artworks in /resource
- Gazebo world or physical setup with accurate artwork placement
- Nodes for control, path planning, and perception active and synchronized

Test 4: Voice-Initiated Navigation to Table

Requirements:

The robot must interpret a verbal command requesting relocation (e.g., “Take me to a table”) and autonomously navigate to a known destination using path planning and control.

Assumptions:

- The table position is known and mapped.
- The voice recognition system is active and calibrated.
- The path planning and control subsystems are functional.
- The robot is in a known starting position.

Procedure:

1. Place the robot at the entrance/start location.
2. Speak the command: **“Take me to a table.”**
3. The voice subsystem interprets the command and forwards it to trigger a navigation goal to the table.
4. The path planner computes a route.
5. The control system guides the robot along the path, avoiding obstacles as needed.
6. Upon reaching the table, the robot announces arrival or logs it.

Pass or Fail Criteria:

Criteria	*Passed*
Voice Command System passes the correct desired table location to path planning subsystem >80% of the time	
Path Planning and Control Subsystems are able to process sent table and successfully simulate robot moving to correct table >80% of the time	

Extra Resources Necessary:

- Functional microphone and voice command processing node
- Known map with at least one table location
- ROS 2 nodes for path planning and control
- Simulated or physical table object at the destination

Test 5: Full Integration

Requirements:

The robot must autonomously respond to a guest's voice command, navigate safely to a destination, avoid dynamic obstacles, recognize and describe nearby artworks, and return to its starting position — integrating all subsystems.

Assumptions:

- The TurtleBot3 is fully operational and initialized in the gallery map.
- All ROS 2 nodes for voice, path planning, control, and perception are running.
- At least one table and artwork are visible in the environment.
- The environment has consistent lighting and minimal noise for speech input.

Procedure:

1. Start robot at the designated gallery entrance.
2. Speak the command: "Take me to a table."
3. Robot uses voice recognition to interpret the command.
4. Path planning generates a route to an available table.
5. The control subsystem follows the path while avoiding dynamic and static obstacles.
6. Upon arrival, robot announces: "We have arrived."
7. User gives next command: "Tell me about this artwork."
8. Robot uses the camera feed to detect and identify the artwork using the perception subsystem.
9. The robot describes the artwork using stored metadata.
10. Robot receives a final command: "Take me back to the entrance."
11. It replans a path and navigates back, completing the interaction.

Pass or Fail Criteria:

Criteria	*Passed*
Passes if the robot completes the full sequence with no manual intervention in >80% of trials in simulation.	
Passes if the robot completes the full sequence with no manual intervention in >60% of trials using a real TurtleBot.	

Extra Resources Necessary:

- Predefined map with labelled tables and artwork positions
- Microphone and speaker system
- Loaded artwork template set
- Voice command trigger phrases
- Obstacle objects to simulate guests

Appendix G: Technical Document Draft

Technical Documentation

Project Overview:

An automatic robotic solution is being developed for the Tom Ugly's Art Gallery's opening art exhibition. The solution will be part of a three-stage system, the Gallery Greeters project will handle the hosting role of the gallery.

Key System Features:

- Greeting Guests at the Doorway
- Leading Guests to Available Tables
- Placing Drink Orders
- Leading Guests to the Artworks
- Wander the Gallery waiting to Assist Guests
- Entertaining Guests with Art Based Fun Facts, Information about Artworks or completing a "Dance"
- Returning to its Charging Station
- Providing Information about Table Capacity and Order Status

The Subsystems:

1. ***Voice Commands***, which handle the direct interaction between the robot and users.
2. ***Path Planning***, which sets safe paths for the robot to travel around the environment.
3. ***Control,*** which handles the velocity commands of the robot as well as obstacle avoidance.
4. ***Perception***, which is used to identify the artworks within the environment.

Requirements

Hardware	Software	Libraries
• TutrleBot3 (camera, lidar and external connectivity access) • Microphone • Digital Screen	• Ubuntu 22.04 • ROS2 • Gazebo • VSCode	• Python • Porcupine from PicoVoice • Google Speech to Text • Pyttsx3 • Pyaudio • Pillow • Screeninfo • Sounddevice • Opencv-python • Cv_bridge • Numpy

Installation

Ros2 Humble install instructions

If Ros2 humble is already installed skip head to cloning the Gallery Greeter repository.

Ensure that the Ubuntu Universe repository is enabled

```
sudo apt install software-properties-common
sudo add-apt-repository universe
```

Add the ROS 2 GPG key

```
sudo apt update && sudo apt install curl -y
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o
/usr/share/keyrings/ros-archive-keyring.gpg
```

Add the repository to your sources list

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-
keyring.gpg] http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo
$UBUNTU_CODENAME) main" | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

Update your apt repository caches after setting up the repositories and ensure your system is up to date before installing new packages

```
sudo apt update
sudo apt upgrade
```

Install the desktop, ROS-Base and development tools

```
sudo apt install ros-humble-desktop
sudo apt install ros-humble-ros-base
sudo apt install ros-dev-tools
```

Source the following file to setup the environment

```
source /opt/ros/humble/setup.bash
```

Clone the Gallery Greeters Repository into your ros2_ws

```
#If you don't have a ros2_ws
mkdir -p ~/ros2_ws/src

cd ~/ros2_ws/src

# Clone the Gallery Greeters repository via SSH
git clone git@github.com:dylan-pk/Gallery-Greeters.git

# OR (if no SSH key is set up)
# git clone https://github.com/dylan-pk/Gallery-Greeters.git
```

Installing Turtlebot3 Packages

Install Gazebo

```
sudo apt install ros-humble-gazebo-*
```

Install Cartographer

```
sudo apt install ros-humble-cartographer
sudo apt install ros-humble-cartographer-ros
```

Install Navigation2

```
sudo apt install ros-humble-navigation2
sudo apt install ros-humble-nav2-bringup
```

For a clean setup, remove all existing Turtlebot installation

```
sudo apt remove ros-humble-turtlebot3-msgs
sudo apt remove ros-humble-turtlebot3
```

Inside the ~/ros2_ws/src directory clone the turtlebot packages

```
Cd ~/ros2_ws/src
git clone -b humble-devel https://github.com/ROBOTIS-GIT/DynamixelSDK.git
git clone -b humble-devel https://github.com/ROBOTIS-GIT/turtlebot3_msgs.git
git clone -b humble-devel https://github.com/ROBOTIS-GIT/turtlebot3.git
git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3_simulations.git
```

INSERT INSTRUCTIONS ON EDITING YAML's and SDF's

Copying environment models, launch files, world files and updates yaml files

An extremely important step in ensuring the world and simulation launches as expected. The models ensure the world gets populated in gazebo physically, the launch files open the gazebo world and spawn the turtlebot with the added functionality of setting the TurtleBot's Z position and orientation. Finally the TurtleBot3 Navigation 2 param yaml file ...

```
cp -r ~/git/Gallery-Greeters/Environment/Models/*
~/ros2_ws/src/turtlebot3_simulations/turtlebot3_gazebo/models/

cp ~/git/Gallery-Greeters/Environment/launch/*.launch.py
~/ros2_ws/src/turtlebot3_simulations/turtlebot3_gazebo/launch/

cp ~/git/Gallery-Greeters/Environment/worlds/Gallery_Test2.world
~/ros2_ws/src/turtlebot3_simulations/turtlebot3_gazebo/worlds/

cp ~/git/Gallery-Greeters/Param/waffle_pi.yaml
~/ros2_ws/src/turtlebot3/turtlebot3_navigation2/param/waffle_pi.yaml
```

Moving the maps to the ros_ws

```
mv ~/git/Gallery-Greeters/Maps/* ~/ros2_ws/
```

Install external libraries

```
pip install pvporcupine SpeechRecognition pyttsx3 pyaudio Pillow screeninfo sounddevice
opencv-python numpy
```

Install ROS bridge pack

```
sudo apt install ros-humble-cv-bridge
```

Build and source the workspace

```
cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash
echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc
```

Pre-Running Code - API Key and path directory

Please note that at the current stage of development, before running the code your own api key for access to Porcupine from PicoVoice requires your own API key which can be obtained from:

https://console.picovoice.ai/

Simply create an account and request a porcupine API key, then copy it from the website and replace the current ACCESS KEY variable in SpeechToText.py with your access key.

Secondly the path directory must be updated in the artbot.launch.py file replacing user in line with your Linux user name and for the real map replacing the real_map_name with your map

```
sim_map_file = os.path.expanduser('/home/<USER>/ros2_ws/gallery_map.yaml')

real_map_file = os.path.expanduser('/home/<USER>/ros2_ws/<real_map_name>.yaml')
```

Running the System

Simulation

Clean Gazebo and Nav2

To ensure the packages and software launches correctly, kill any Gazebo or Rvis processes using the commands below. Doing so will allow the map to launch freshly each time and ensure correct localisation.

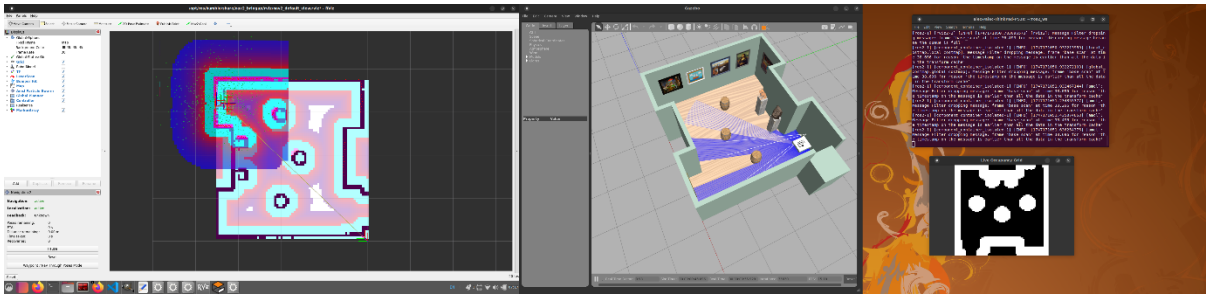
```
killall -9 gzserver gzclient
pkill –f map_server
pkill –f nav2
pkill –f rviz2
```

Run the Gallery Greeters Launch File

Launching the artbot.launch.py file using the commands below will open up the Gazebo world with the Turtlebot spawned into its charging port. It will load the map and then run the control, path planning and perception packages. All will be active and ready to interact when voice commands are given and the path planning package will automatically localise the artbot within the map so it is ready to plan it's path. The live occupancy grid will also open showing a black background of free space and the known objects in white which will update as new objects are detected.

```
ros2 launch gallery_greeters2 artbot.launch.py use_simulation:=true
```

Once Simulation is successfully launched and localisation has been completed you can launch the artbot interface when best suited for you.



Run Robot Interface Natively

To run the artbot interface and interact with the TurtleBot via voice commands simply run the following,

```
Ros2 run package_audio_messages speech_node
```

Running this command will launch the artbot's display as seen below and will be ready to receive voice commands. see Interacting with the Robot.

Real TurtleBot

Setting up the environment

The first step to running the Gallery Greeters artbot in a real environment is to set up the physical environment. Within the walls of the gallery, mark out the starting position, known obstacles/tables and the artworks along the gallery walls like seen below.

Connecting to the turtlebot

To establish a connection to the TurtleBot your computer must be connected to the same network, for example our testing was connected to a network called TurtleBot MMR.

Once both the TurtleBot and your computer are connected to the same network, connect with it via SSH. This requires knowing the IP address of the TurtleBot if this is unknown boot into the

TurtleBot system with a monitor and keyboard and run the following command which will return your IP address in the terminal and note the username of the pi for the SSH connection.

```
hostname -I
```

Set the artbot up into the starting position of the environment and connect the battery to power on in the home position.

Ping the IP address until it returns that it is active.

```
Ping <IP address>
```

Once the artbot is able to be connected with then ssh into the raspberry pi.

```
ssh <pi username>@<IP address>

For example
ssh ubuntu@192.168.0.213
Then enter the password of the TurtleBot
```

Then launch the artbot through this command using a unique domain ID .

```
export  ROS_DOMAIN_ID = <ID>

For example

export ROS_DOMAIN_ID = 44

ros2 launch turtlebot3_bringup robot.launch.py
```

Repeat the SSH process in another terminal to activate the camera on the TurtleBot, instead of the artbot launch command run this command.

```
export ROS_DOMAIN_ID = <ID>

ros2 run camera_ros camera_node --ros-args -p format:=MJPEG -p width:=640 -p height:=480
```

Mapping the environment

With the SSH and TurtleBot active now run through the mapping process as outlined below running each command in a new terminal.

Launch nav2

```
ros2 launch nav2_bringup navigation_launch.py use_sim_time:=True
```

Launch the slam toolbox

```
ros2 launch slam_toolbox online_async_launch.py use_sim_time:=True
```

Bring up Rvis for visualising the map

```
ros2 run rviz2 rviz2 -d /opt/ros/humble/share/nav2_bringup/rviz/nav2_default_view.rviz
```

Then Teleop in the environment building up the map file, be sure not to collide with anything!

```
ros2 run turtlebot3_teleop teleop_keyboard
```

Once the map is built to an acceptable level save the map with your desired map name

```
ros2 run nav2_map_server map_saver_cli -f <map name>
```

Below is an example of the map generated with the physical environment from before.

Now reset the TurtleBot to its starting position severing the connection with the computer and powering it back on in the starting position. Teleop the artbot to each of the artworks and tables gathering the coordinates of each known position and orientation keeping the artbot far away enough from the artworks so that they are in frame of the camera, this is roughly 30cm away from the artwork when using A4 paper.

Then update the artwork locations within package_audio_messages using the following command and input the names of the artworks along with the locations and orientation that the artbot needs to be in for the camera to percieve the artwork.

```
cd ros2_ws/src/gallery_greeters/package_audio_messages/package_audio_messages/

python3 ArtDataInput.py
```

Run the artbot (physical)

Rerun the connection to the TurtleBot with the artbot powered on in the home position. Then in a new terminal run the launch file.

```
export ROS_DOMAIN_ID =<ID>

ros2 launch gallery_greeters2 artbot.launch.py use_simulation:=false
```

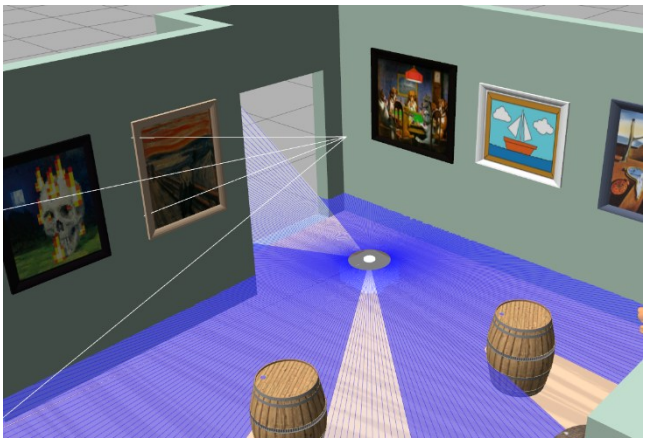
Interacting with the Robot

To initialise the interaction start by calling out to the artbot "Hey **Jarvis**"

There are 10 ways to interact with Jarvis as an operator or guest of the art gallery:

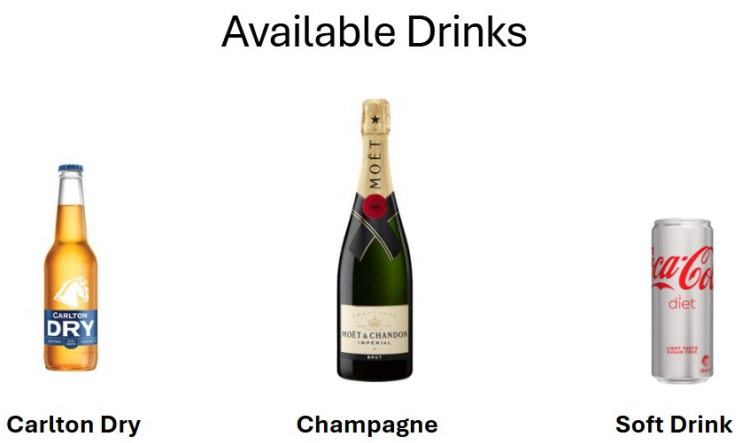
1. To tell Jarvis to go greet guests say "Go into **Greeting** mode"

   To which Jarvis will move to his set position for greeting guests.



2. To place a drinks order with Jarvis say "I would like a **drink**"

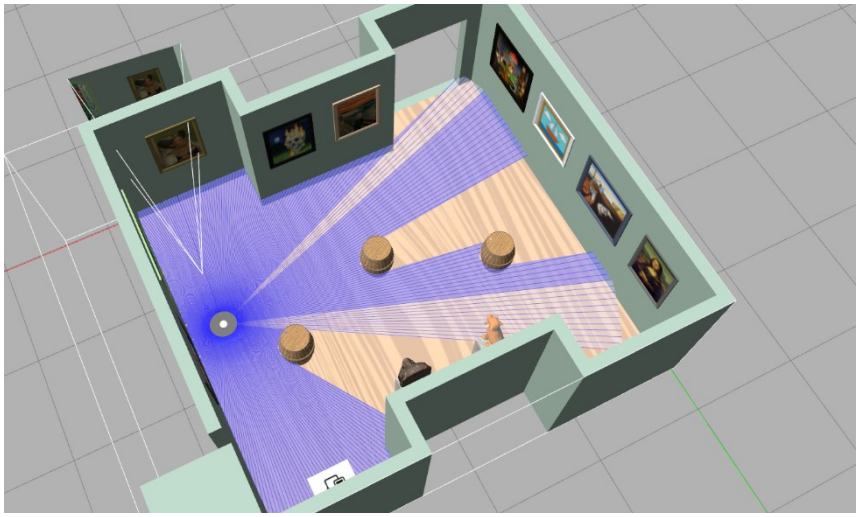
   To which Jarvis will respond asking what drink you would like displaying the options for you. Jarvis will recognise the type of drink you ask for if it is on the list and follow up asking for the quantity of drinks you would like. After confirming these details, Jarvis will ask if you want any more drinks until you say no. The order will then be sent to the kitchen and wait staff robots.



3. To ask about the art say "I would like to look at the **art**" or "Tell me about the **artwork**"
   To which Jarvis will ask "Which Artwork would you like to know about"

   a. Respond with a simplified name such as "the one with **clocks**", "The one from **Minecraft**" or "The **Duchess**" and the artbot will take you to it.

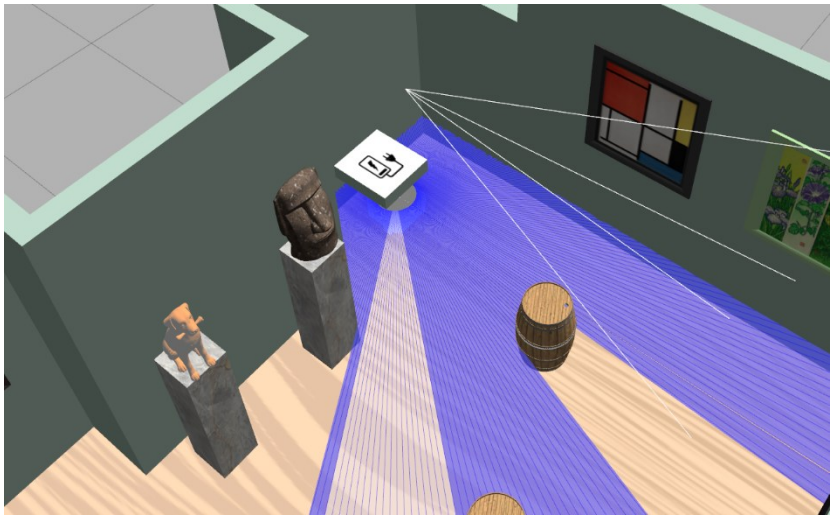
4. To put Jarvis in Sentry Mode (Moving around the environment waiting for guest interaction) say "Go for a **walk**"
   To which Jarvis will plan a path around the perimeter of the gallery and continue



   that path until another "Hey Jarvis" command is given.

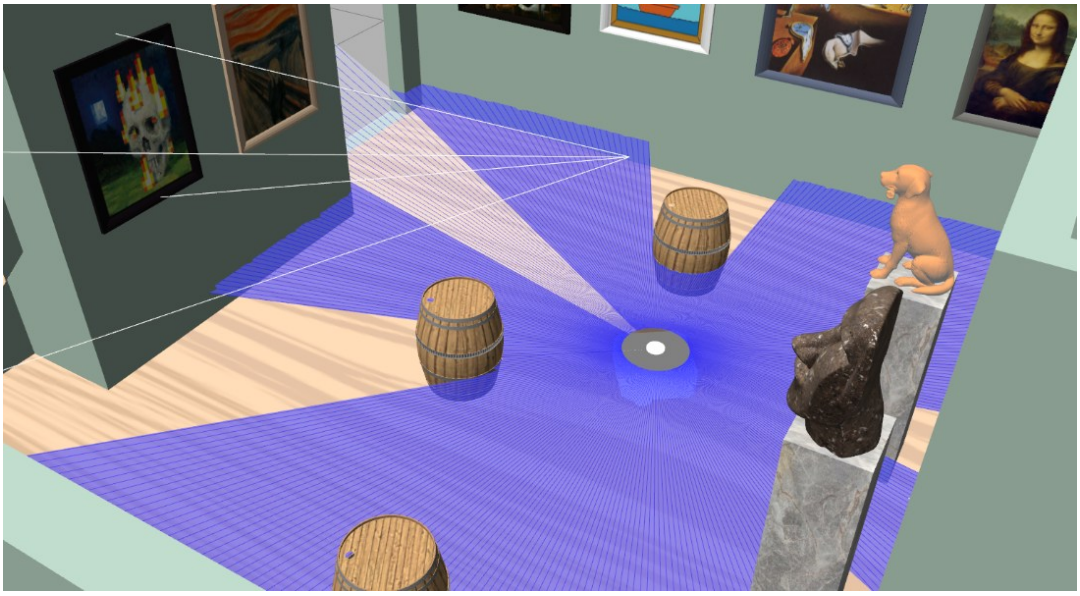
5. To get Jarvis to dance say "Do a **Dance"**
   To which Jarvis will dance on the spot

6. To send Jarvis back to the charging port say "Go and **charge**"



To which Jarvis will go to his Initial position at his charging port.

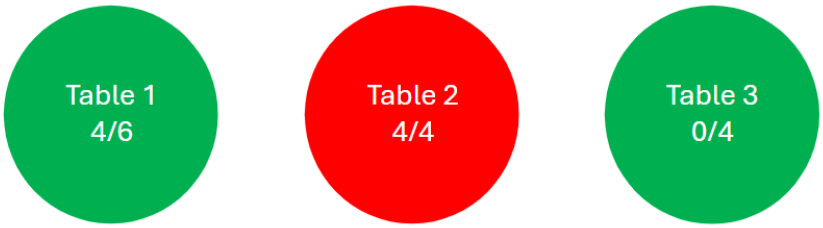
7. To get Jarvis to take you to a table say "Take me to a **Table**"
   To which Jarvis will ask which table and after giving him a number from 1-3 he will move to one of the 3 available tables.



8. To call a waiter to the table say "Call a **waiter**" which will bring a waiter to your current location or the closest

9. To ask Jarvis for a fun fact say “Tell me a fun **fact**”
   To which Jarvis will respond with one of his many fun facts

10. To receive an update about the status of the tables say “Tell me the **table status”**
    Jarvis will then display an infographic visually depicting the status of all tables



11. To be taken through each artwork, ask Jarvis ”Take me to a **Painting**” and Jarvis will take you to an artwork without you needing to ask which artwork it

Subsystem

Voice Commands

Purpose

The Voice Command subsystem exists to facilitate interaction between guests and the system in a user friendly way. This subsystem allows for the system to be very mobile and for users to have no technical experience in order to use the system's full functionality.

Key Topics/Services/Files

- The Voice Command Subsystem primarily uses 2 files
    - ***SpeechToText.py* – This is the file contains the node and the main method and controls most of the audio recognition code. The only code for the available commands is that which requires multiple prompts and responses for information.**
    - ***commands.py*** *- this is the file which holds most of the command functioning and utilizes most of the publishers, subscribers and services*
- Key ROS Topics and Services:
    - ***client_artIdentification*** – Used in SpeechToText.py to prompt the template_match package to retrieve the name or the painting infront of the robot to then provide information to guests
    - ***goal_pub*** – Used by functions in commands.py to send locations to the path_planning package to be translated into movement for various functions.
    - ***mode_pub*** – Used by functions in commands.py to send the mode to the control system to run different specialised movement scripts or function

Configurable Settings

In SpeechToText.py at the top there is a Global Variable called TESTING_MODE:
If set to True the system does not function based on the google speech to text to get commands and instead takes a user inputed integer when prompted.

In SpeechToText.py at the top there is a Global Variable called ACCESS_KEY:
An access key can be gained by making an Picovoice account (https://console.picovoice.ai/) and must be updated for each new user to run the system.

Path Planning

Purpose

The purpose of the path planning subsystem is to calculate a valid, collision free path from the TurtleBot's current position to either a given goal or vector of goals, depending on operating mode. The path planning subsystem uses an A* algorithm and a pre-generated map of the environment to generate the path between the given locations.

Key Topics/Services/Files

Source Files:

- **path_to_goal4.cpp** - for real robot
- **Path_to_goal5.cpp** - for simulated robot

Subscribed Topics:

- /pose_topic — Receives the goal pose (with orientation).
- /odom — Receives artbot odometry for current pose, used with /pose_topic to generate path.
- /robot_mode — Controls behaviour switching (e.g. navigation, patrol, dance).
- /sentry_path — Receives a vector of goals for use in the sentry patrol made.
- /interrupt_signal — Triggers path cancellation and halts motion.
- /map — Receives SLAM updates that are used to update the dynamic occupancy grid
- /updated_occupancy_grid — Receives dynamic obstacles to update the path planning occupancy grid
- /goal_pose – Receives single goal pose for direct navigation.

Published Topics:

- /computed_path — Publishes the generated path for the Control subsystem to follow.
- /known_static_obstacles — Publishes known static obstacles from the occupancy grid
- /static_occupancy_grid — Publishes the initial inflated occupancy grid
- /initialpose — Publishes initial pose for localisation

Service Topic

- /request_static_grid — Processes requests from other nodes to receive the static occupancy grid

Configurable Settings

Settings	Value	Description
Inflation radius	Inflation_radius_m = **0.15**	Radius (m) of area that known obstacles are inflated to avoid collisions
Static inflation radius	static_inflation_radius_m = **0.7**	Radius (m) of area that known obstacles are inflated to scale dynamic inflation radius
Dynamic inflation radius	dynamic_inflation_radius_m = **static_inflation_radius_m / 3.0**	Dynamic radius (m) of area that known obstacles are inflated to avoid collisions
Map	map_meta_ = load_map_yaml("**real_map5.yaml**")	The file (yaml) the map is loaded from
Path replanning threshold	if (std::hypot(dx, dy) < **0.2**)	Threshold to avoid path replanning if Turtlebot has reached goal
Behaviour Modes	/robot_mode = • **0**: Default • **1**: Sentry Mode • **2**: Dance routine • **3**: Rotate and perform visual detection	Path planning is performed differently depending on the mode: • **0**: Navigate to a single given goal • **1**: Navigate through a vector of known goals • **2**: No path planning • **3**: No path planning

Control

Purpose

The controller governs the artbot's motion using PID control for both linear and angular adjustments. It dynamically tracks waypoints along a path and performs behavior switching based on robot mode. The system also collaborates with ObstacleAvoidance to detect and handle dynamic obstacles, ensuring updated path planning.

Key Topics/Services/Files

Source Files:

- p_controller.hpp / .cpp — Core control logic, motion execution, and mode handling.
- obstacle_avoidance.hpp / .cpp — LaserScan + local costmap-based dynamic obstacle detection and occupancy grid updates.

Subscribed Topics:

- /odom – Receives artbot odometry for pose tracking.
- /computed_path – Subscribes to the planned path.
- /robot_mode – Controls behavior switching (navigation, sentry, dance, scan).
- /interrupt_signal – Triggers path cancellation and halts motion.
- /pose_topic – Receives final goal pose (with orientation).
- /scan – LaserScan data used to detect dynamic obstacles.
- /local_costmap/costmap – Receives local costmap for detecting new obstacles.
- /static_occupancy_grid – Receives the static map used as a baseline reference.

Published Topics:

- /cmd_vel – Issues velocity commands for linear and angular movement.
- /reaches – Publishes a Bool indicating the artbot has reached the final goal.
- /updated_occupancy_grid – Publishes dynamic occupancy grid highlighting unknown obstacle regions.

Service Clients:

- /request_static_grid — One-shot service used at startup to retrieve a static occupancy grid for map alignment.

Configurable Settings

Settings	Value	Description
Control Gains	Kp_angular = 0.6; Ki_angular = 0.0; Kd_angular = 0.5; Kp_linear = 1.0; Ki_linear = 0.0; Kd_linear = 0.1;	Proportional gain for distance-based movement. Proportional gain for heading correction.
Tolerance	tolerance = 0.05	Distance threshold for goal arrival.
Velocity Clamps	Max linear velocity: 0.12 m/s Min linear velocity: 0.03 m/s	Min and Max velocity for artbot to move at.
Behavior Modes (via /robot_mode)	0: Navigate to artwork 1: Sentry Mode 2: Dance routine 3: Rotate and perform visual detection	Types of movement modes that the artbot can enter.

Perception

Purpose

The purpose of the perception subsystem is to identify visual features within the environment, with a primary focus on detecting and differentiating paintings affixed to the gallery walls. This is achieved through template matching techniques, allowing the robot to visually confirm the presence and identity of specific artworks, which enables behaviour switching and information display in response to user queries.

Key Topics/Services/Files

Source Files:

- template_match.py – Implements all perception logic and service handling for template matching
- /template_matching/resources/ - contains “png” and “jpeg” files required to perform perception matching

Subscribed Topics:

- /camera/image_raw – Receives live image stream from the TurtleBot3’s camera for processing

Published Topics:

- /interrupt_signal – Publishes a bool message to notify the system of a successful match that may interrupt current behaviour

Service Topics:

- /perform_template_matching – process requests to perform template matching using most recent camera image

Published Files (For Debugging Purposes):

- /tmp/latest_camera_frame.jpg – Stores the latest received image frame
- /tmp/match_<label>.jpg – Stores visualised keypoint matches for each template comparison to the last captured frame

Configurable Settings

Settings	Value	Description
Confidence Threshold self.declare_parameter('confidence_threshold', *VALUE*)	0.65	Minimum confidence required to accept a match result
ORB Features self.orb = cv.ORB_create(nfeatures=*VALUE*)	1000	Number of ORB features extracted from templates and camera from
Top Match Count top_n = *VALUE*	20	Number of top keypoint matches considered for scoring
Distance Normalization D_max = *VALUE*	75	Normalization to scale average keypoint match distance

Operation Details

The node loads all image templates from the resource/ folder on startup.

- Upon receiving a service request, the system performs ORB feature matching between the current camera image and all available templates.
- For each template, the top 20 matches are evaluated to compute an average distance, which is normalized into a confidence score.
- A debug image is saved showing matched keypoints for each comparison (/tmp/match_<label>.jpg).
- The most recent raw image is also saved to /tmp/latest_camera_frame.jpg for verification and analysis.
- Although the /interrupt_signal publisher is initialized, it is currently commented out in the handle_template_matching function.

Troubleshooting & FAQs

Issues installing pyaudio

If there are build issues installing pyaudio try running the following command

```
sudo apt-get install portaudio19-dev python3-pyaudio
```

Map not showing up in Rvis

Make sure you have corrected the file path on line 20 with your linux user name

```
map_file = os.path.expanduser('/home/<USER>/ros2_ws/gallery_map.yaml')
```

Also ensure that you have killed all Gazebo and Rvis processes before running the launch file

```
killall -9 gzserver gzclient

pkill –f map_server
pkill –f nav2
pkill –f rviz2
```

RQT Image View issues

When running the template matching launch file and attempting to view image data through the rqt_image_view window the data isn't displayed. This issue is bypassed by running the following command. This will continue to open the latest frame if the defualt image viewer app is set to the built in linux image viewer.

```
xdg-open /tmp/latest_camera_frame.jpg
```

Appendix H: GitHub Excerpts

Screenshots taken from the Gallery Greeters repository.

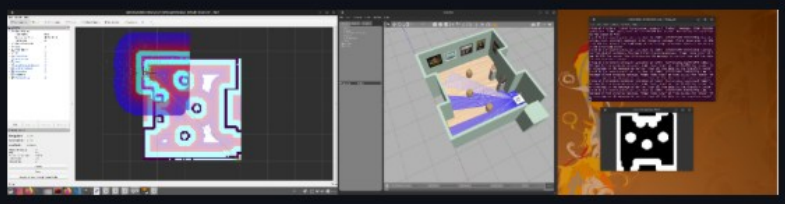
.vscode	Everything works as desired in test...	2 months ago
Environment	cleaning the git	last week
Maps	pres maps	last week
Packages	d	last week
Param	cleaning the git	last week
ReadMeImages	added mapping photo	last week
path_planning/src/turtlebot_...	fix	
README.md	Update README.md	

Name	Last commit message	Last commit date
..		
gallery_greeters2	d	last week
pController	final	last week
package_audio_messages	final	last week
template_match	cleaning the git	last week
turtlebot_nav	final submission	last week

Run the Gallery Greeters Launch File

Launching the artbot.launch.py file using the commands below will open up the Gazebo world with the Turtlebot spawned into its charging port. It will load the map and then run the control, path planning and perception packages. All will be active and ready to interact when voice commands are given and the path planning package will automatically localise the robot within the map so it is ready to plan it's path. The live occupancy grid will also open showing a black background of free space and the known objects in white which will update as new objects are detected.

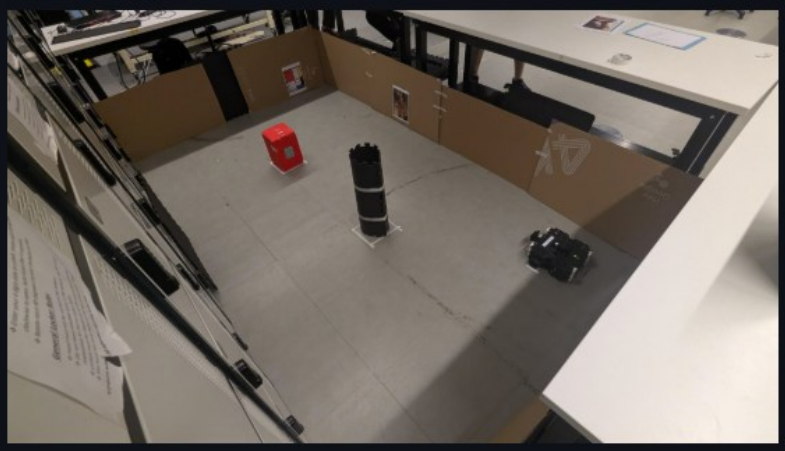
```
ros2 launch gallery_greeters2 artbot.launch.py use_simulation:=true
```



Running The System - Real Robot

Setting Up the Environment

The first step to running the Gallery Greeters artbot in a real environment is to set up the physical environment. Within the walls of the gallery, mark out the starting position, known obstacles/tables and the artworks along the gallery walls like seen below.



Connecting to The Turtlebot

To establish a connection to the TurtleBot your computer must be connected to the same network, for example our testing was connected to a network called TurtleBot MMR.

Once both the TurtleBot and your computer are connected to the same network, connect with it via SSH. This requires knowing the IP address of the TurtleBot if this is unknown boot into the TurtleBot system with a monitor and keyboard and run the following command which will return your IP address in the terminal and note the username of the pi for the SSH connection.